Group Project

Date Matching Application

Due: Monday, May 04, 2026 at 11:55PM

Group Assignment

Note: This project is a group project and groups should consist of at least 2 persons and no more than 5 persons.

If you have not already done so, please ensure that you let me know by email (laurie.leitch@uwi.edu) who your group members are as soon as is possible.

Background

In this group project, you will develop a DATING APPLICATION web platform (called DriftDater) that allows registered users to create detailed profiles, discover compatible matches, and initiate connections with other users. The application will be built using Vue 3 frontend framework and Flask backend API, with a database to store user profiles and matching information.

This project will reinforce key concepts learned in INFO3180 including:

- Relational database design
- RESTful API development
- Authentication and authorization
- Frontend-backend integration
- User interface/user experience design
- Testing and deployment procedures

You may use the Starter code at: https://github.com/uwi-info3180/info3180-vuejs-flask-starter if you need to.

Role Distribution: Each team must define the following roles:

- ✓ Project Manager (Oversees timeline, coordinates team efforts)
- ✓ Backend Lead (Manages API, database, security)
- ✓ Frontend Lead (Manages UI/UX, Vue 3 components)
- ✓ QA/Testing Lead (Manages testing, validation, documentation)
- ✓ Deployment Lead (Manages deployments, configurations) [if 5 members]

Note: Team members may have overlapping responsibilities; however, each member must contribute meaningfully to the codebase with visible commits and documented contributions.

PROJECT REQUIREMENTS

A. CORE FEATURES (Mandatory)

1. USER AUTHENTICATION & PROFILE MANAGEMENT
   a) User registration with email validation
   b) Secure login/logout functionality
   c) Password hashing (bcrypt or similar)
   d) User profile creation and editing
   e) Profile fields must include:
      i) Basic info (name, age, bio/description)
      ii) Location/geographic preferences
      iii) Interests/hobbies (minimum 3)
      iv) Profile picture upload capability
      v) Additional custom fields (minimum 2)
   f) Profile visibility controls (public/private)

2. MATCHING SYSTEM
   a) Simple matching algorithm based on:
      i) Location proximity (within specified radius)
      ii) Age range preferences
      iii) Shared interests
      iv) At least one additional matching criterion
   b) Display matched profiles to logged-in users

c) Like/Dislike/Pass functionality
d) Match notifications and confirmations
e) Mutual match detection (both users like each other)
f) Browse mode with filtering options

3. USER CONNECTIONS & MESSAGING
a) Simple messaging system for matched users
b) Display conversation list
c) Send and receive messages
d) Message persistence (stored in database)
e) Real-time or near-real-time message updates (Vue reactivity)
f) View message history

4. SEARCH & DISCOVERY
a) Search functionality by:
i) Location
ii) Age range
iii) Interests
iv) Additional criteria
b) Filter applied matches
c) Sort options (newest, most similar, etc.)
d) Save favorite/bookmarked profiles

B. TECHNICAL REQUIREMENTS

Backend (Flask API):

✓ RESTful API design with proper HTTP methods

✓ Database schema with minimum 4-5 normalized tables

✓ Authentication using sessions

✓ Input validation and error handling

✓ CORS configuration for frontend communication

✓ Database migrations (using Flask-Migrate)

Frontend (Vue 3):

✓ Component-based architecture with reusable components

✓ State management (Vuex or Pinia recommended)

✓ Routing (Vue Router for page navigation)

✓ Responsive design (mobile-friendly)

✓ Form validation and user feedback

✓ Loading states and error handling

Database:

✓ SQLAlchemy ORM models

✓ Proper relationships (one-to-many, many-to-many)

✓ Indexes on frequently queried columns

✓ Data integrity constraints

C. OPTIONAL FEATURES (Choose minimum 2)

These features will enhance your grade but are NOT required:

□ Advanced matching algorithm (machine learning basics, collaborative filtering)

□ Premium user features (verified badges, advanced filters, boost profile)

□ User ratings/reviews system

□ Notification system (email or in-app notifications)

□ Location-based services (map integration for nearby matches)

□ Admin dashboard for site moderation

□ Dark mode/theme customization

□ Mobile app (React Native or Flutter)

□ Video chat integration for matched users

□ Comprehensive analytics and user insights

□ Two-factor authentication (2FA)

□ Report/block user functionality

□ Deployment to cloud platform (AWS, Heroku, Render)

SETUP INSTRUCTIONS:

1. Clone the starter repository
2. Create a Python virtual environment: python -m venv venv
3. Activate: source venv/bin/activate (Linux/Mac) or .\venv\Scripts\activate (Windows)
4. Install dependencies: pip install -r requirements.txt
5. Create database: flask db init, flask db migrate, flask db upgrade
6. Start backend: flask --app app --debug run (default: http://localhost:5000)
7. In new terminal, start frontend: npm install && npm run dev (default: http://localhost:5173)

DELIVERABLES

All deliverables must be submitted via GitHub repository with clear documentation.

1. SOURCE CODE (50 points)

1. Complete, well-documented source code
2. Clean, readable code following PEP 8 (Python) and Vue 3 best practices
3. All features properly implemented
4. Version control history (meaningful commits from all team members)
5. No hardcoded credentials or sensitive data
6. .gitignore properly configured
7. requirements.txt and package.json up-to-date

2. DATABASE DESIGN (10 points)

1. ER diagram showing all tables and relationships
2. Database schema documentation
3. Proper normalization (minimum 3rd Normal Form)
4. Indexes on appropriate columns
5. Migration scripts working correctly

3. USER DOCUMENTATION (10 points)

1. README.md with:
    a. Project description
    b. Team member names and roles
    c. Setup instructions (clear, step-by-step)
    d. API documentation (endpoints, parameters, responses)
    e. Deployed application URL (if applicable)
    f. Known issues/limitations
2. User manual/Feature walk-through

4. PRESENTATION (10 points)

1. Live demonstration of all features
2. Code walk-through (10-15 minutes)
3. Architecture and design decisions explained
4. Challenges faced and solutions implemented
5. Team member contributions clearly identified

ASSESSMENT CRITERIA DETAILS

FUNCTIONALITY ASSESSMENT:

The application must fulfill these specific criteria:
[Authentication & Profiles]

✓ Users can register with unique email

✓ Users can log in securely

✓ Users can create and edit profiles

✓ Password reset functionality (optional but recommended)

✓ Profile photo uploads working

✓ All profile required fields present

[Matching System]

✓ Algorithm matches users based on criteria stated above

✓ Matched users are displayed properly

✓ Like/Dislike/Pass buttons function correctly

✓ Mutual matches are detected and confirmed

✓ Match count displayed accurately

✓ Filtering/sorting options work as intended

[Messaging]

✓ Messages can be sent between matched users

✓ Messages persist in database

✓ Conversation history displays correctly

✓ Real-time updates when available

✓ Timestamps show on messages

[Search & Discovery]

✓ Search filters work correctly

✓ Results display accurately

✓ Performance acceptable (< 2 second load time)

CODE QUALITY ASSESSMENT:

Backend (Flask):

✓ Follows REST naming conventions

✓ Proper HTTP status codes returned

✓ Error handling with meaningful messages

✓ Input validation on all endpoints

✓ No N+1 query problems

✓ Efficient database queries

✓ Code comments where necessary

✓ Functions have single responsibility

Frontend (Vue 3):

✓ Components are small and reusable

✓ State management is clear and organized

✓ Proper use of lifecycle hooks

✓ Responsive design tested on multiple screen sizes

✓ Loading states and error messages shown

✓ No memory leaks (proper cleanup)

IMPORTANT GUIDELINES

1. ACADEMIC INTEGRITY

- ✓ All code must be original work from your team
- ✓ External libraries are acceptable (and encouraged)
- ✓ Using code from tutorials/Stack Overflow must be cited
- ✓ Do not use existing dating app codebases without permission
- ✓ Plagiarism will result in zero for project and academic review

2. SECURITY CONSIDERATIONS

- ✓ Never commit API keys, passwords, or secrets to GitHub
- ✓ Use .env files for configuration
- ✓ Validate all user input
- ✓ Hash passwords using bcrypt or similar
- ✓ Implement CORS properly
- ✓ Use HTTPS for any deployed application
- ✓ Protect against SQL injection (use ORM)

3. DEPLOYMENT (Optional but Recommended)

Suggested platforms:

a) Heroku (free tier available)
b) AWS (free tier for students)
c) Render (free tier available)

4.

PRESENTATION

✓ All team members must present

✓ Prepare a live demo (backup screenshot if internet fails)

✓ Be ready to answer technical questions

✓ Demonstrate all features working

✓ Show code and explain design decisions

✓ Discuss challenges and solutions

5. SUBMISSION CHECKLIST

Before submitting, ensure:

☐ All code pushed to GitHub

☐ README.md is complete and clear

☐ Database setup instructions work

☐ No sensitive data in repository

☐ All dependencies in requirements.txt and package.json

☐ Application runs without errors on fresh installation

☐ Git history shows all team members' contributions

☐ Tests pass locally

☐ Documentation is complete

☐ Presentation slides prepared

Start early, ask questions, and support your team members. Good luck!

Submission

Submit your code via the "Group Project Submission" link on VLE. You should submit the following links:

1. Your Github repository URL for your Flask app e.g. https://github.com/{yourusername}/info3180-project
2. Your URL for your Render app e.g. https://{yourappname}.render.org

Appendix

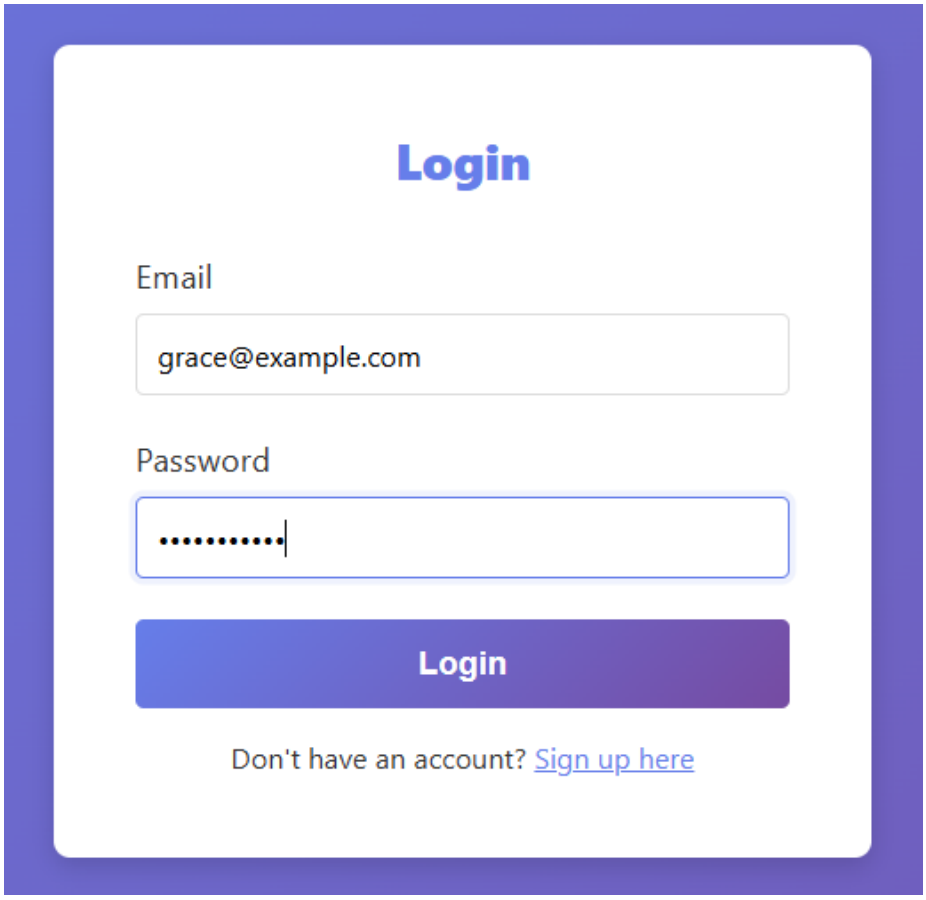

FIGURE 1. HOME PAGE (NOTE: YOUR WEBPAGE DOES NOT HAVE TO LOOK EXACTLY THE SAME BUT MUST HAVE THE MENU OPTIONS TO LOGIN, REGISTER AND VIEW REPORTS)

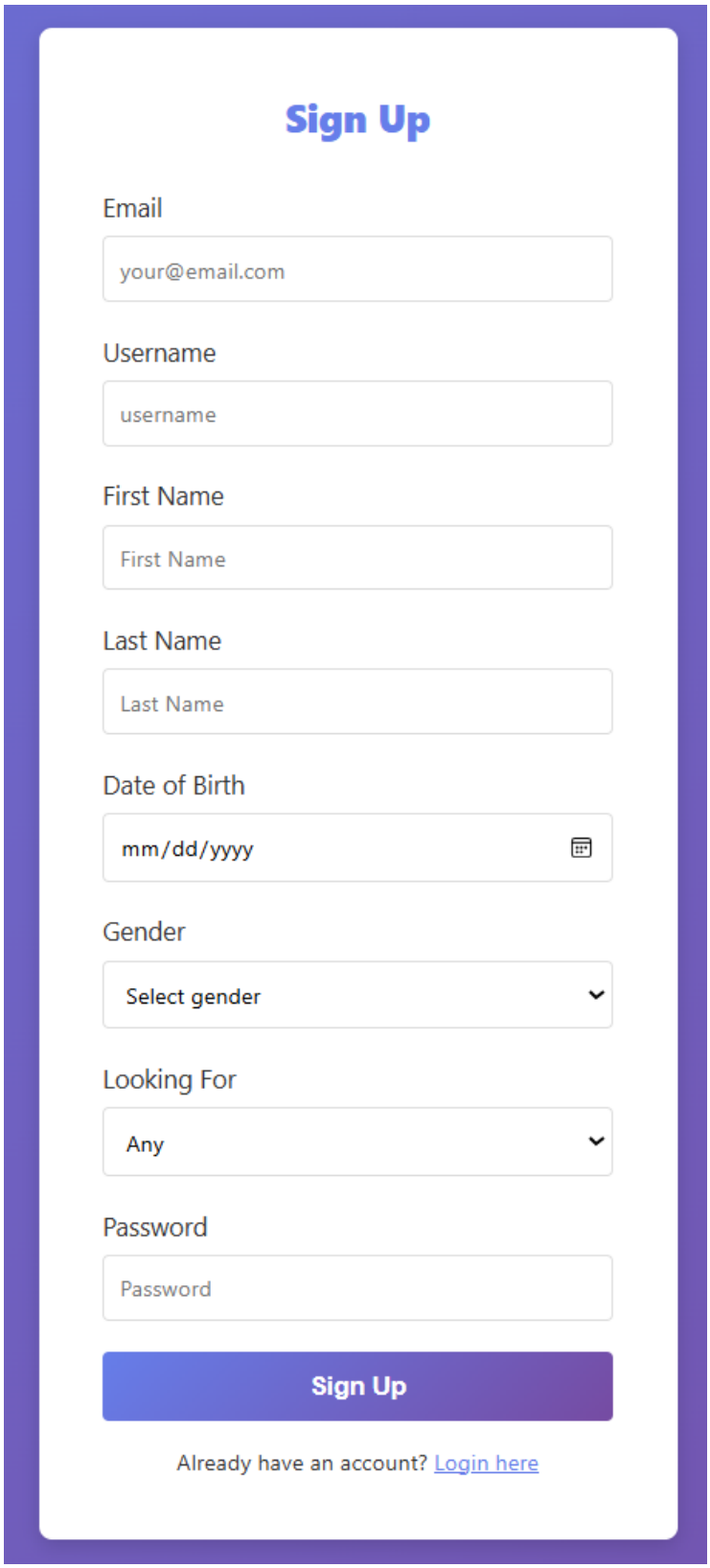

FIGURE 2. USER REGISTRATION/SIGN-UP

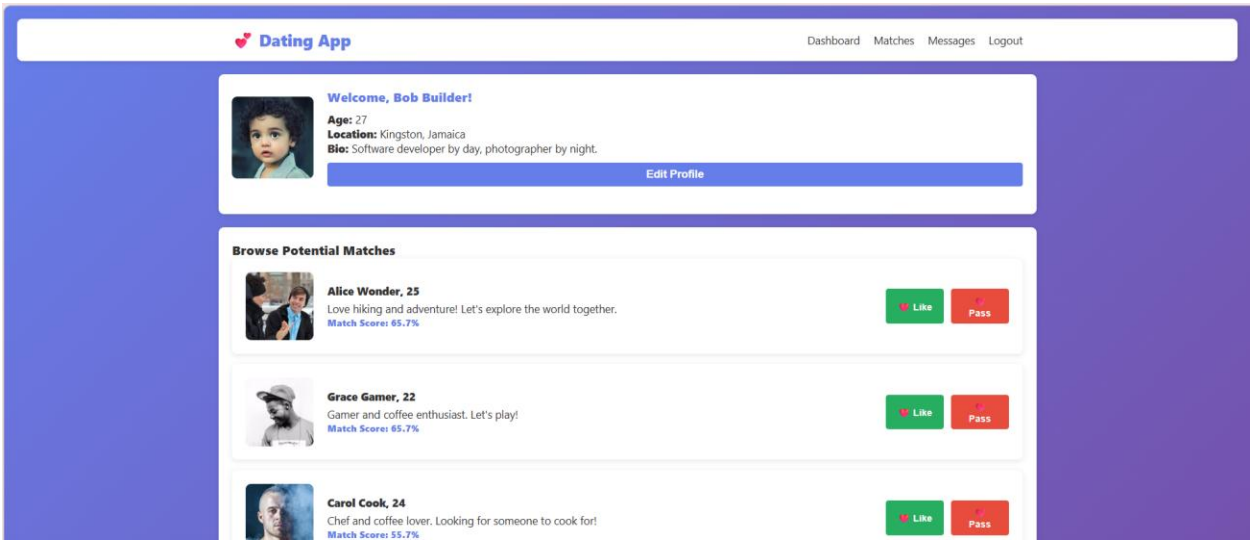

FIGURE 3: USER PAGE. THIS DISPLAYS THE LATEST ADDED PROFILES THAT MATCH YOURS

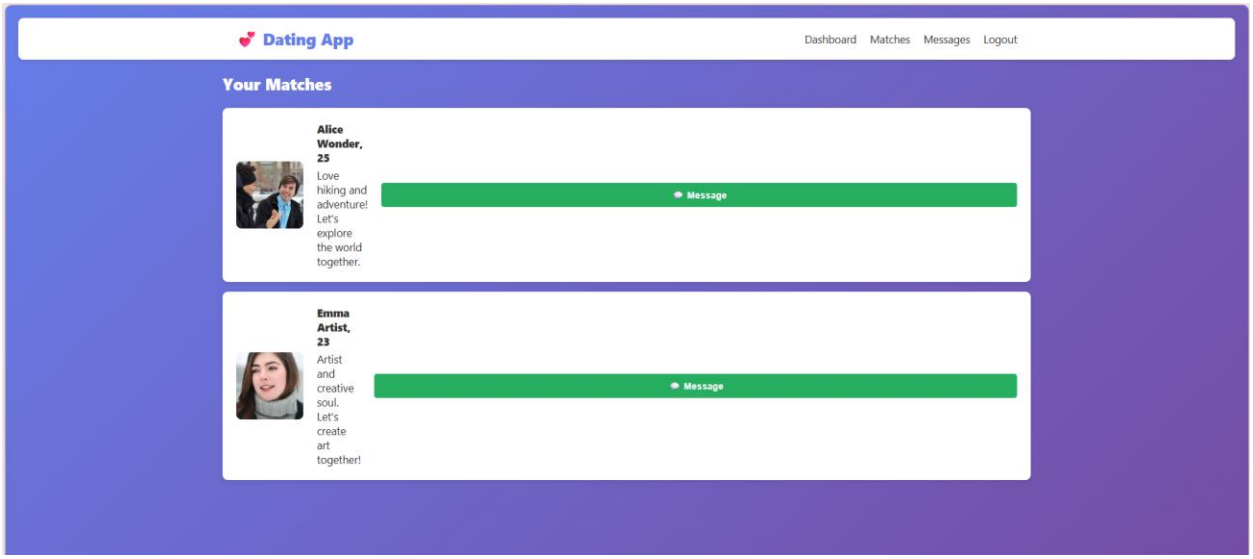

FIGURE 4: USER MATCHES BASED ON MATCHING FUNCTIONS AS WELL AS WHEN TWO USERS LIKE EACH OTHER BY CLICK ON LIKE FOR THE PROFILES.

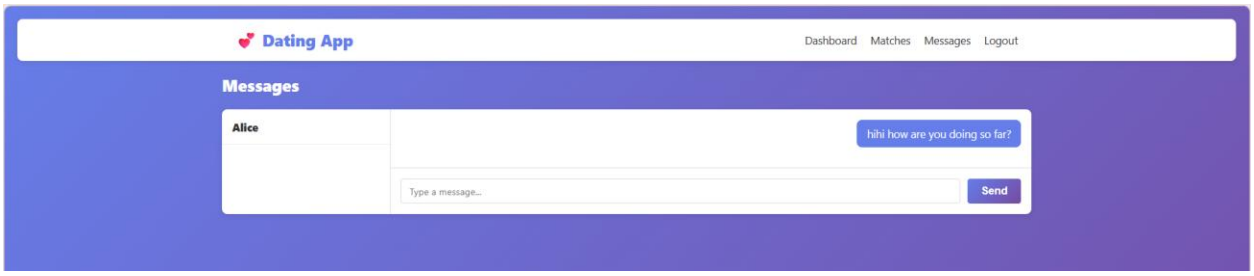

FIGURE 5: USERS CAN SEND MESSAGES ONLY TO OTHER USERS WHO THEY LIKED AND HO LIKE THEM

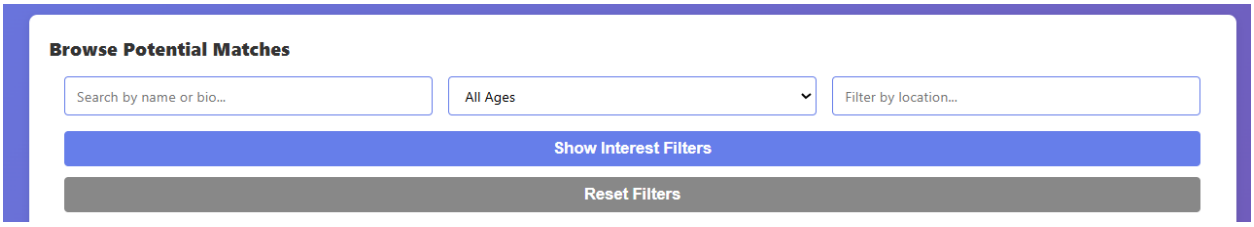

FIGURE 6: SEARCHING AND FILTERING SHOULD OCCUR ON YOUR LISTS ESPECIALLY WHEN THERE ARE A NUMBER OF ITEMS IN THE LIST.

==========
RESOURCES & SUPPORT
==========

Recommended Resources:

Backend (Flask):
- **Flask official documentation: https://flask.palletsprojects.com/**
- **SQLAlchemy ORM: https://docs.sqlalchemy.org/**
- **Flask-SQLAlchemy: https://flask-sqlalchemy.palletsprojects.com/**

Frontend (Vue 3):
- **Vue 3 documentation: https://vuejs.org/**
- **Vue Router: https://router.vuejs.org/**
- **Pinia (State Management): https://pinia.vuejs.org/**
- **Vue CLI/Vite: https://vitejs.dev/**

Database:
- **PostgreSQL: https://www.postgresql.org/**
- **SQLite: https://www.sqlite.org/**

==========
KEY DATES
==========

Week 1: **Project planning, team formation, architecture design**
Week 2-3: **Database design and API development**
Week 4-5: **Frontend development**
Week 6-7: **Polish and optional features**
Week 8: **Final testing, documentation, presentation prep**

Presentation Date: [To Be Announced]

==========
CONTACT & CLARIFICATIONS
==========

For questions or clarifications regarding this project:
Contact: Mr Laurie Ivor Leitch (876-288-2126)
Email: laurie.leitch@uwi.edu
Office: UWI, Mona MITS

Please reach out early if you:
- **Are unsure about requirements**
- **Encounter technical difficulties**
- **Have team conflicts**
- **Need an extension**
- **Plan to attempt advanced features**

==========
FINAL NOTES
==========

This is a capstone project that brings together everything learned in this course. The goal is not just to build a dating app, but to demonstrate proficiency in:

- **Full-stack web development**
- **Database design and optimization**
- **API development and design**
- **Frontend development with modern frameworks**
- **Testing and quality assurance**
- **Team collaboration and project management**
- **Professional documentation and communication**